ACE

AI Coding Environment

PRODIGY9

2026-03-01

prodigy9.co

AI Coding Environment

Automated Claude Environments

The Problem

Every AI coding session starts with the same ritual:

Every AI coding session starts with the same ritual:

- *"Which model? Where's the API key?"*

Every AI coding session starts with the same ritual:

- *"Which model? Where's the API key?"*
- *"What conventions does this codebase follow?"*

Every AI coding session starts with the same ritual:

- *"Which model? Where's the API key?"*
- *"What conventions does this codebase follow?"*
- *"What skills does the agent need to do its job?"*

Every AI coding session starts with the same ritual:

- *"Which model? Where's the API key?"*
- *"What conventions does this codebase follow?"*
- *"What skills does the agent need to do its job?"*

You answer these questions every. single. time.

```
$ ace setup prod9/school   # once per repo
$ ace                      # every time after
```

```
$ ace setup prod9/school   # once per repo
$ ace                      # every time after
```

ACE eliminates the setup tax.

What ACE Does

1. Check environment readiness

Git? Backend installed? Credentials valid?

1. Check environment readiness

Git? Backend installed? Credentials valid?

2. Install and update skills

Clone school, symlink conventions, keep everything current.

1. Check environment readiness

Git? Backend installed? Credentials valid?

2. Install and update skills

Clone school, symlink conventions, keep everything current.

3. Configure and launch

Inject prompt, set credentials, exec the backend.

You type

```
$ ace setup prod9/school
```

Once per repo.

```
$ ace
```

Every time after.

ACE does

1. Clone the school repo
2. Authenticate with LiteLLM
3. Symlink skills into project
4. Merge configs (`ace.toml` + school)
5. Inject session prompt
6. exec the backend

All transparent. Sub-second.

Define a **school** — one repo of shared skills for your org.

Define a **school** — one repo of shared skills for your org.

Everyone **syncs on every run**. No drift.

Define a **school** — one repo of shared skills for your org.

Everyone **syncs on every run**. No drift.

Zero manual setup across machines.
New hire? `ace setup` and they're ready.

Define a **school** — one repo of shared skills for your org.

Everyone **syncs on every run**. No drift.

Zero manual setup across machines.
New hire? `ace setup` and they're ready.

Edit skills in-place → **propose changes back** to the school.

ACE is **not** the AI.

ACE is **not** the AI.

ACE is the **launcher**.

- Backends are pluggable: **Claude Code**, **OpenCode**, more to come
- ACE prepares the environment, then calls exec
- After exec — ACE is gone from memory

ACE is **not** the AI.

ACE is the **launcher**.

- Backends are pluggable: **Claude Code**, **OpenCode**, more to come
- ACE prepares the environment, then calls exec
- After exec — ACE is gone from memory

The backend runs the show. ACE just made sure it had everything it needed.

Business Value

Without ACE

- Each dev configures AI differently
- "What conventions do we follow again?"
- New hire spends days on setup
- Tribal knowledge lives in someone's head
- "Works on my machine" — for AI too

With ACE

- **One school** — identical AI behavior, every dev

Without ACE

- Each dev configures AI differently
- "What conventions do we follow again?"
- New hire spends days on setup
- Tribal knowledge lives in someone's head
- "Works on my machine" — for AI too

With ACE

- **One school** — identical AI behavior, every dev
- **One command** — new hire runs `ace setup`, done

Without ACE

- Each dev configures AI differently
- "What conventions do we follow again?"
- New hire spends days on setup
- Tribal knowledge lives in someone's head
- "Works on my machine" — for AI too

With ACE

- **One school** — identical AI behavior, every dev
- **One command** — new hire runs `ace setup`, done
- **Skills are code** — tribal knowledge survives turnover

Without ACE

- Each dev configures AI differently
- "What conventions do we follow again?"
- New hire spends days on setup
- Tribal knowledge lives in someone's head
- "Works on my machine" — for AI too

With ACE

- **One school** — identical AI behavior, every dev
- **One command** — new hire runs `ace setup`, done
- **Skills are code** — tribal knowledge survives turnover
- **Auto-sync** — no config drift between machines

Visibility — skills are code-reviewed PRs. You see exactly what AI is told.

Visibility — skills are code-reviewed PRs. You see exactly what AI is told.

Control — conventions and guardrails ship org-wide through the school.

Visibility — skills are code-reviewed PRs. You see exactly what AI is told.

Control — conventions and guardrails ship org-wide through the school.

Consistency — same skills, same inputs → comparable output quality.

Visibility — skills are code-reviewed PRs. You see exactly what AI is told.

Control — conventions and guardrails ship org-wide through the school.

Consistency — same skills, same inputs → comparable output quality.

Cost — ramp-up for new projects and new hires drops to `ace setup`.

Encode

Domain knowledge becomes a **school**.
Best practices, conventions, guardrails —
code-reviewed `SKILL.md` files.

Expertise that lived in someone's head — now it ships.

Sell

Publish a school. Customers run `ace setup`.
Instant access to your methodology.

Consulting firms, agencies, platform teams — expertise as a subscription.

Philosophy

The naive approach

Copy skills into each repo.
Check them in. Ship it.

Feels right on day one.

Day thirty

The naive approach

Copy skills into each repo.
Check them in. Ship it.

Feels right on day one.

Day thirty

One typo fix in a skill.
14 repos to patch.
3 are already diverged.

Copying is easy. Keeping copies in sync is the real problem.

Single source

School = git repo
Skills live once
Cache on each machine

No copies to drift.

Edit in place

Symlinks point to cache
Edit in any project
`ace school propose` back

Contribution is free.

Instant onboard

`ace setup prod9/school`
Clone, link, done
Same skills as everyone

Convention over configuration.

Traditional: pin a version, freeze behavior, prevent breakage.
Assumes the consumer is **dumb**.

Traditional: pin a version, freeze behavior, prevent breakage.
Assumes the consumer is **dumb**.

An LLM is not dumb. It **reads**. It **adapts**.
Same prompt, different day — different output anyway.

Traditional: pin a version, freeze behavior, prevent breakage.
Assumes the consumer is **dumb**.

An LLM is not dumb. It **reads**. It **adapts**.
Same prompt, different day — different output anyway.

The consumer is intelligent. Always-latest is cheap.

ACE = file sync + config merge + exec

ACE = file sync + config merge + exec

No skill resolution. No dependency graph. No runtime logic.
ACE is gone after exec.

ACE = file sync + config merge + exec

No skill resolution. No dependency graph. No runtime logic.
ACE is gone after exec.

The best orchestrator does the least orchestration.

Schools & Skills

```
prod9/school/
├── school.toml
├── CLAUDE.md
└── skills/
    ├── general-coding/
    │   └── SKILL.md
    ├── rust-coding/
    │   └── SKILL.md
    └── typst-coding/
        └── SKILL.md
```

A **git repo** that defines how AI works for your org.

```
prod9/school/
├── school.toml
├── CLAUDE.md
└── skills/
    ├── general-coding/
    │   └── SKILL.md
    ├── rust-coding/
    │   └── SKILL.md
    └── typst-coding/
        └── SKILL.md
```

A **git repo** that defines how AI works for your org.

`school.toml`

Name, imports, school-level config.

```
prod9/school/
├── school.toml
├── CLAUDE.md
└── skills/
    ├── general-coding/
    │   └── SKILL.md
    ├── rust-coding/
    │   └── SKILL.md
    └── typst-coding/
        └── SKILL.md
```

A **git repo** that defines how AI works for your org.

`school.toml`

Name, imports, school-level config.

`skills/`

Shared skill packages — symlinked into every project.

```
prod9/school/
├── school.toml
├── CLAUDE.md
└── skills/
    ├── general-coding/
    │   └── SKILL.md
    ├── rust-coding/
    │   └── SKILL.md
    └── typst-coding/
        └── SKILL.md
```

A **git repo** that defines how AI works for your org.

`school.toml`

Name, imports, school-level config.

`skills/`

Shared skill packages — symlinked into every project.

`CLAUDE.md`

Global instructions applied to all projects.

A skill is a **`SKILL.md`** with frontmatter + instructions.

```
rust-coding/SKILL.md

---
name: rust-coding
description: >
  Rust conventions — error handling,
  Option/Result idioms, serde, testing.
---
```

A skill is a **`SKILL.md`** with frontmatter + instructions.

```
rust-coding/SKILL.md

---
name: rust-coding
description: >
  Rust conventions — error handling,
  Option/Result idioms, serde, testing.
---
```

AI reads skills **contextually** — not config, not templates.
Prose that the AI understands.

From the community

```
$ ace import anthropics/skills \
    --skill commit
```

skills.sh — Anthropic-blessed community registry.

How it works

- No `npx skills` — **`ace import`** handles it

From the community

```
$ ace import anthropics/skills \
    --skill commit
```

skills.sh — Anthropic-blessed community registry.

How it works

- No `npx skills` — **`ace import`** handles it
- Imported into the **school repo**

  Whole team gets it.

From the community

```
$ ace import anthropics/skills \
    --skill commit
```

skills.sh — Anthropic-blessed community registry.

How it works

- No `npx skills` — **`ace import`** handles it
- Imported into the **school repo**
  Whole team gets it.
- Tracked in `school.toml` imports
  Code-reviewed.

Architecture

Config

Pure I/O. No merging.
No logic. No opinions.

```
ace.toml
school.toml
user_config
```

State

Domain tree.
Merge configs, resolve schools,
run actions.

Install, Update, Link,
Authenticate, Exec

Ace

Entrypoint.
Holds State + output mode,
drives the CLI.

Config ← **State** ← **Ace**

Discover

1. Find configs
2. Parse & merge

`ace.toml` + school + user prefs → single state tree.

Prepare

3. Authenticate
4. Fetch school
5. Sync skills
6. Check tooling

Clone or update, symlink, verify prerequisites.

Launch

7. Select backend
8. Inject prompt
9. exec

ACE replaces itself with the backend. Gone from memory.

What’s Next

PKCE auth flow — multi-user rollout blocker.

PKCE auth flow — multi-user rollout blocker.

Auto-detect school

`prod9/ace` → `prod9/school`, zero-config.

`prod9/ace` → `prod9/school`, zero-config.

More backends — OpenCode, Aider, Cursor. Same school, any tool.

`prod9/ace` → `prod9/school`, zero-config.

More backends — OpenCode, Aider, Cursor. Same school, any tool.

School marketplace — discover and import from the community.

ace

prodigy9.co